VIBLIX

MICRO PROJECT REPORT
submitted by

MOHAMED SUHAIL D

(CEM24CS045)

SREERAG SR

(CEM24CS065)

REEMA RS

(CEM24CS051)

RISANA THASEER R
(CEM24CS053)

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of the Degree
of
Bachelor of Technology in
Computer Science and Engineering

Department of Computer Science and Engineering

College of Engineering Muttathara

Thiruvananthapuram

OCTOBER 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING MUTTATHARA

VISION

To generate competent professionals in Computer Science and Engineering by imparting quality education and there by facilitating cutting edge research and development to serve as valuable resources for the industry and society.

MISSION

- Provide learning ambience through strong theoretical and practical background with an emphasis on software development.
- Establish industry interaction programmes to enhance the technical-knowhow at par with the current trends in the industry and to promote the entrepreneurship skills.
- Promote research-based projects in the emerging areas of Computer Science and Engineering.
- Empowering the youth in rural communities with computer education.
- Inculcating professional behaviour, leadership qualities, team spirit, skills on problem solving, critical thinking, and ethical responsibilities.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING MUTTATHARA

CERTIFICATE

This is to certify that the micro project work report entitled "**VIBLIX"** submitted by **MOHAMED SUHAIL D(CEM24CS045),SREERAG SR(CEM24CS065),REEMA RS(CEM24CS051),RISANA THASEER R(CEM24CS053)** to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the subject PBCST304 Object Oriented Programming in Computer Science and Engineering is a bonafide record of the micro project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Guide:	**HEAD OF THE DEPT:**
Sowmya k S	**Dr. K S Angel Viji**
Assistant professor	**Associate Professor**
Department of CSE	**Department of CSE**

DECLARATION

We undersigned hereby declare that the micro project report entitled "**VIBLIX"**, submitted for partial fulfillment of the requirements of the Third Semester micro project for the subject PBCST304 Object oriented Programming, prescribed by APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the guidance of Assistant Professor, Ms.Sowmya K S.

The submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Muttathara

MOHAMED SUHAIL D
SREERAG S R
REEMA R S
RISANA THASEER R

ACKNOWLEDGEMENT

We take this opportunity to express our deep sense of gratitude and sincere thanks to all who helped us to complete the work successfully. Our first and foremost thanks goes to God Almighty who showered in immense blessings on our effort. We wish to express our sincere thanks to Dr.V Praseedalekshmi, (Principal, College of Engineering Muttathara) for providing us with all the necessary facilities and support. We would like to express our sincere gratitude to Dr. K S Angel Viji (Head of the department) , for there support and co-operation. We wish to express our sincere gratitude towards our guide Mrs. Sowmya K S for giving advices to work with the project and complete it successfully. We wish to express our sincere gratitude towards all the teaching and non teaching staff members of our Department. Finally, we thank our parents, all our friends, near and dear ones who directly and indirectly contributed to the success of this work.

MOHAMED SUHAIL D

SREERAG SR

REEMA RS

RISANA THASEER R

ABSTRACT

Viblix is an Object-Oriented Programming (OOP) based entertainment tracking system designed to help users organize and analyze their viewing habits across anime, TV series, and movies. The application allows users to create personalized watchlists, track episode progress, write reviews, and view detailed analytics such as total watch time and completion rates. Built using modular OOP principles, Viblix ensures scalability, reusability, and maintainability of code.The system provides recommendations based on user preferences and supports multiple users with separate watchlists. Data persistence is achieved through file handling and database integration using JDBC and MySQL. With a user-friendly interface developed in JavaFX/Swing and a backend implemented in Java (Spring Boot/Servlets), Viblix offers a seamless experience for managing entertainment consumption. Future enhancements include social sharing, API integration with popular streaming platforms, and AI-based recommendations.Overall, Viblix demonstrates the practical application of OOP concepts to build an efficient, extensible, and real-world software solution for modern entertainment tracking.

CONTENTS

Contents Page No.

ACKNOWLEDGMENT …….. i

ABSTRACT …….. ii

LIST OF FIGURES ……. iii

Chapter 1 : INTRODUCTION ……. 1

1.1 OBJECTIVE ……. 1

1.2 MOTIVATION …….. 2

1.3 PROBLEM STATEMENT …… 2

Chapter 2 : SYSTEM REQUIREMENTS AND SPECIFICATIONS …… 3

2.1 FUNCTIONAL REQUIREMENTS.....4

2.2 NON-FUNCTIONAL REQUIREMENTS.....5

2.3 HARDWARE REQUIREMENTS 6

2.4 SOFTWARE REQUIREMENTS.....6

Chapter 3: METHODOLOGY AND DESIGN.....8

3.1 INTRODUCTION.....8

3.2 USE CASE DIAGRAM.....8

3.3 CLASS DIAGRAM 9

3.4 DATA FLOW DIAGRAM.....9

3.5 ACTIVITY DIAGRAM 10

3.6 SEQUENCE DIAGRAM.....11

3.7 DEVELOPMENT MODEL 12

3.8 SYSTEM DESIGN 12

3.9 SYSTEM ARCHITECTURE.....12

3.10 DESIGN CONSIDERATIONS.....13

3.11 DATABASE DESIGN.....13

Chapter 4 : IMPLEMENTATION..... 14

4.1 IMPLEMENTATION 14

4.2 IMPLEMENTATION OVERVIEW.....14

4.3 CODE.....15

Chapter 5: TESTING 19

5.1 INTRODUCTION.....19

5.2 OBJECTIVES OF TESTING.....19

5.3 TYPES OF TESTING PERFORMED 20

5.4 TEST CASE TABLE.....20

Chapter 6: RESULT 21

6.1 USER INTERFACE.....22

6.2 RESULTS AND DISCUSSION.....25

Chapter 7: CONCLUSION 26

LIST OF FIGURES

No.	Title	Page No
4.3.1	User class code	15
4.3.2	Database connectivity code	15
4.3.3	Login code	16
6.1.1	Login page	22
6.1.2	Welcome Interface	22
6.1.3	Dashboard Interface	23
6.1.4	Add show/series function in action	24
6.1.5	Update function in action	24
6.1.6	Analytics function in action	25

CHAPTER 1

INTRODUCTION

In today's digital world, people consume entertainment from multiple platforms such as Netflix, Crunchyroll, and YouTube. However, keeping track of what we watch, our progress, and the time spent often becomes difficult. Viblix is developed to solve this problem by providing a unified system that helps users organize, track, and analyze their anime, TV shows, and movie-watching habits.The project is designed using Object-Oriented Programming (OOP) principles to ensure modularity, scalability, and reusability. Users can create personalized watchlists, mark progress, rate shows, and generate detailed viewing analytics. The system also offers recommendations based on viewing preferences and stores all data persistently through file handling and database integration.Viblix demonstrates the practical use of OOP concepts in real-world software development by combining simplicity, functionality, and analytical insights into one efficient entertainment management platform.

1.1 OBJECTIVE

The main objective of Viblix is to design and develop an interactive and user-friendly entertainment tracking system using Object-Oriented Programming (OOP) principles. The project aims to help users manage their anime, TV shows, and movie watchlists efficiently while analyzing their viewing patterns.

Specific objectives include:

To implement core OOP concepts such as encapsulation, inheritance, and polymorphism for modular design.

To provide features for adding, editing, and tracking watchlist entries. To

generate viewing analytics and personalized recommendations.

To ensure data persistence through file handling and database integration.

To create a scalable system that can be expanded with social and API-based features in the future

1.2 MOTIVATION

In today's streaming era, people watch content across multiple platforms such as Netflix, Disney+, and Crunchyroll. With so many choices, it becomes difficult to remember which shows have been completed, which are in progress, or how much time has been spent watching. This lack of organization often leads to confusion and missed content.

The idea behind Viblix came from the common experience of wanting a single place to track all viewing activities. As programming students, we were inspired to create a system that not only simplifies entertainment tracking but also demonstrates the power of Object-Oriented Programming (OOP) concepts in solving real-world problems. Viblix serves as both a practical tool for viewers and a valuable learning project that applies modular design, data persistence, and user interaction in a meaningful way.

1.3 PROBLEM STATEMENT

With the rise of multiple streaming platforms, users often struggle to keep track of the shows and movies they watch. Many viewers forget which episodes they've completed, what they plan to watch next, or how much time they've spent watching overall. Existing platforms usually track progress only within their own service and lack a unified system to manage all entertainment data in one place.

To address this issue, Viblix aims to provide a centralized entertainment tracking system that allows users to maintain watchlists, monitor viewing progress, analyze total watch time, and receive personalized recommendations — all through a modular and scalable application built using Object-Oriented Programming (OOP) principles.

CHAPTER 2

SYSTEM REQUIREMENTS AND SPECIFICATIONS

Viblix is an Object-Oriented entertainment tracking system designed to help users organize, monitor, and analyze their anime, TV series, and movie-watching habits. The system allows users to create and manage personalized watchlists, update episode or season progress, write ratings and reviews, and view detailed analytics such as total watch time and completion rates.

The system architecture follows a three-layer structure:

1. Frontend (Client-Side): Developed using JavaFX/Swing, providing an interactive and user-friendly interface for users to manage their watchlists.

2. Backend (Server-Side): Built with Java (using Servlets or Spring Boot) to handle business logic, user requests, and data processing.

3. Persistence Layer (Database): Uses MySQL for storing user data, watchlists, ratings, and analytics, connected via JDBC for seamless communication.

- This layered design ensures modularity, scalability, and easy maintenance. By combining strong OOP principles with efficient data management, Viblix offers a complete and reliable solution for entertainment tracking and analysis.

2.1 FUNCTIONAL REQUIREMENTS

1. User Registration & Login

Users can create accounts and log in to access their personal watchlist

2. Add to Watchlist

Users can add new series, movies, or anime to their watchlist.

3. Edit Watchlist Entries

Users can update watched episodes, change progress status, or edit details.

4. Status Management

Ability to mark shows as:

Watching

Completed

Dropped

Plan to Watch

5. Ratings & Reviews

Users can rate content (1–10 scale) and write short reviews.

6. Analytics Generation

System calculates and displays:

Total hours watched

Completion-rate

Most-watched genres

7. Recommendations

Suggest new titles similar to high-rated or favorite genres.

8. Multi-user Support

Each user maintains a separate watchlist and analytics data.

9. File Handling / Data Persistence

Watchlists, reviews, and analytics are saved and loaded from files or a database (MySQL).

2.2 NON- FUNCTIONAL REQUIREMENTS

1. Performance

Application should run smoothly even with 1000+ entries stored.

2. Usability

Interface (menu or GUI) should be simple and intuitive for all users.

3. Reliability

Analytics and recommendations must produce accurate results consistently.

4. Maintainability

Code should follow OOP principles (classes, inheritance, encapsulation) for easy updates or scaling.

5. Portability

Must run on any system with Java or Python installed (cross-platform compatibility).

6. Scalability

Should support future expansion such as API integration or cloud sync.

7. Data Integrity

Ensure that stored watchlist and analytics data are not corrupted or lost between sessions.

8. Security

Basic user authentication to prevent unauthorized access to watchlists.

2.3 HARDWARE REQUIREMENTS

Processor: Intel i3 or higher

• RAM: 4 GB or more

• Storage: 100 MB free

• Display: Any standard console output

2.4 SOFTWARE REQUIREMENTS

1. Operating System:

The project can run on any modern operating system such as Windows, Linux, or macOS. Cross-platform compatibility ensures that the system is portable and accessible to a wide range of users.

2. Programming Language:

The application is developed using Java or Python, both of which fully support Object-Oriented Programming (OOP) principles such as inheritance, encapsulation, and polymorphism.

3. Integrated Development Environment (IDE):

Development and testing can be performed using popular IDEs such as Eclipse, IntelliJ IDEA, VS Code, or PyCharm, depending on the chosen programming language.

4. Database Management System:

MySQL is used as the database to store user data, watchlists, ratings, and analytics. It ensures persistence, reliability, and efficient data retrieval.

5. Frontend Framework:

The graphical user interface is developed using JavaFX or Swing, providing an interactive and user-friendly experience.

6. Backend Framework:

The backend logic is implemented with Java Servlets or Spring Boot, handling user requests, data processing, and analytics generation.

7. Version Control (Optional):

A GitHub repository can be used for version control, team collaboration, and tracking updates throughout the development process.

.

CHAPTER 3

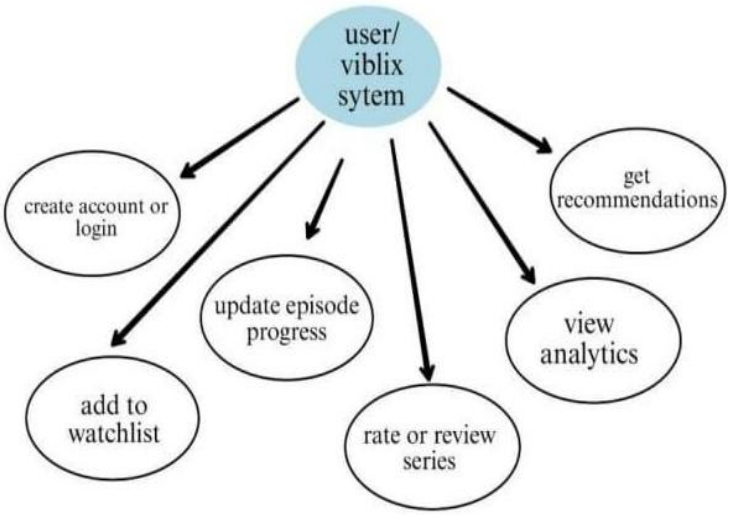
METHODOLOGY AND DESIGN

3.1 INTRODUCTION

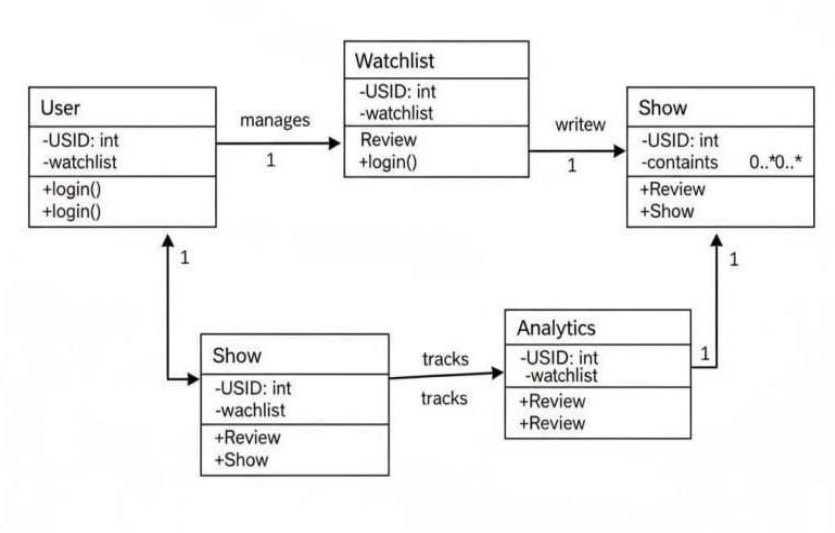
This chapter describes the methodology adopted in the development of VIBLIX and the system design that guided its implementation. The methodology ensures that the system is built efficiently, adhering to Object-Oriented Programming (OOP) principles, and that the design models clearly represent the structure and functionality of the application.

3.2 USE CASE DIAGRAM

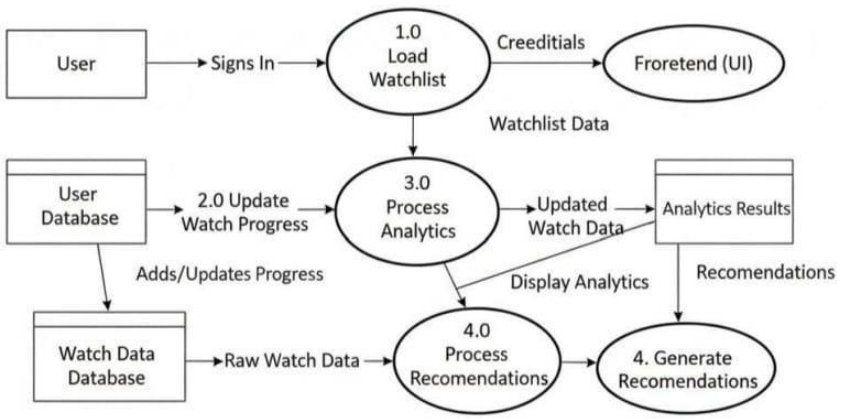
- The user is the actor in this use case Diagram
- The actions create account, add to watchlist ,update progress, view analytics



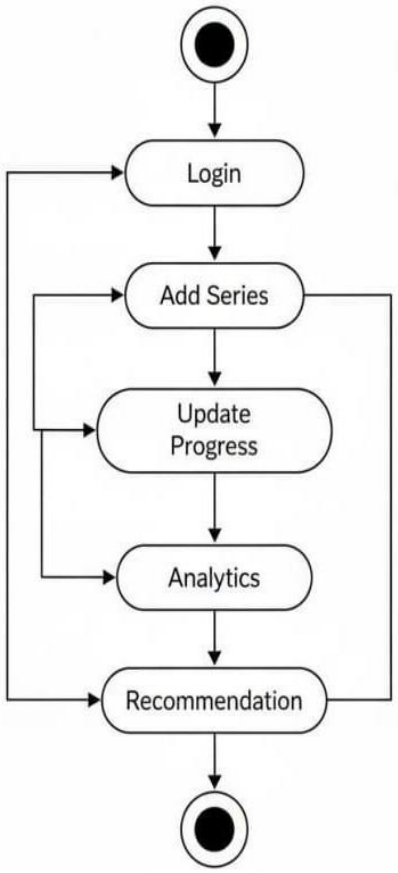
3.3 CLASS DIAGRAM



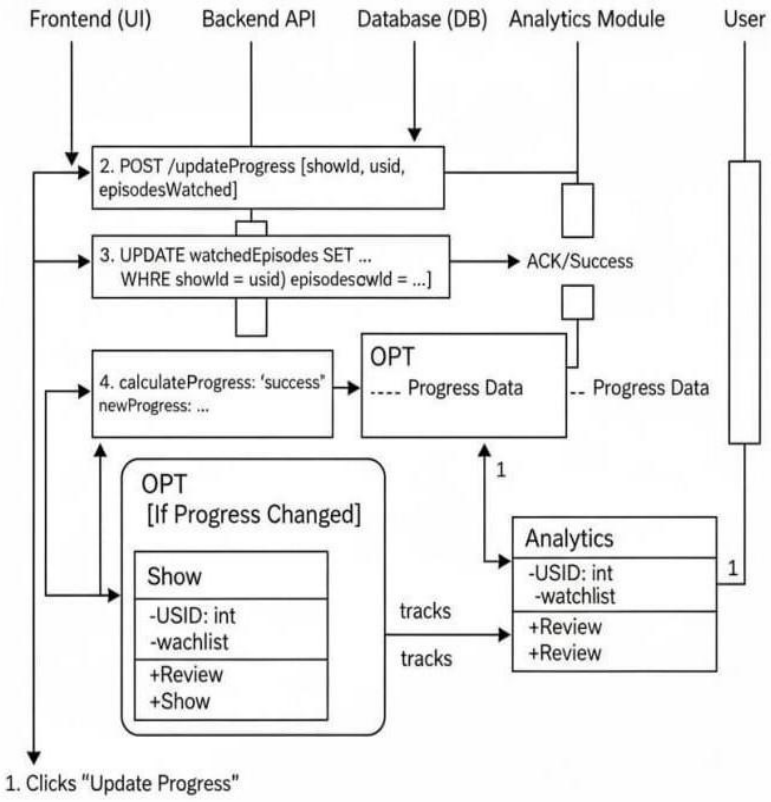
3.4 DATA FLOW DIAGRAM



3.5 ACTIVITY DIAGRAM



3.6 SEQUENCE DIAGRAM



3.7 DEVELOPMENT MODEL

The Incremental Development Model was used.

Each module (User Management, Watchlist, Analytics, Recommendations) was implemented and tested separately, allowing early validation and feedback before full integration.

3.8 SYSTEM DESIGN

System design defines the structure, components, interfaces, and data flow of VIBLIX. The design focuses on modularity, ease of maintenance, and extensibility.

3.9 SYSTEM ARCHITECTURE

The system is structured into three main layers:

1. Frontend Layer (Client-Side):

Developed using JavaFX or Swing for GUI.

Handles user interactions and displays watchlists, analytics, and recommendations.

2. Backend Layer (Server-Side):

Implemented in Java (Spring Boot/Servlets).

Manages core logic, data processing, and communication with the database.

3. Persistence Layer (Database):

Uses MySQL for data storage.

Handles watchlist data, ratings, user information, and analytics.

4. Communication:

Established using JDBC (Java Database Connectivity) between the backend and database.

3.10 DESIGN CONSIDERATIONS

Modularity: Each module performs a single function for easy maintenance.

Reusability: Classes and methods can be reused in future versions or integrations.

Scalability: Architecture supports future additions like API integration and social features.

Data Security: User data is protected through encapsulation and controlled file/database access.

3.11 DATABASE DESIGN

The database design of VIBLIX defines how all user, watchlist, review, and analytics data are stored and managed efficiently.

It follows a relational database model implemented using MySQL, ensuring data consistency and integrity across all modules.

The design includes key entities such as User, Show, Watchlist, Review, and Analytics, each represented as separate tables connected through primary and foreign keys.

This structured approach supports key system features like user authentication, content tracking, rating management, and performance analytics.

Normalization techniques are applied to minimize redundancy and improve data reliability, while relational links ensure smooth interaction between different components of the system.

Overall, the database design ensures that VIBLIX is efficient, scalable, and capable of handling large volumes of user data securely.

CHAPTER 4

IMPLEMENTATION

4.1 IMPLEMENTATION

The implementation phase involves transforming the system design into a functional and executable program.

In VIBLIX, this stage focuses on writing the actual code based on the object-oriented design, integrating modules, and connecting the user interface with the backend database.

The goal of this phase is to develop a fully working version of the system that performs all the required functions — from managing user watchlists to generating analytics and recommendations.

4.2 IMPLEMENTATION OVERVIEW

The VIBLIX system was implemented using Java as the core programming language, adhering to Object-Oriented Programming (OOP) principles.

The graphical interface was developed using JavaFX/Swing, and the backend communication was handled through JDBC (Java Database Connectivity) for interaction with the MySQL database.

This layered implementation ensures modularity, scalability, and ease of maintenance.

4.3 CODE

```java
package model;

public class User {
    private int id;
    private String name;
    private String email;
    private String password;

    public User(int id, String name, String email, String password){
        this.id = id; this.name = name; this.email = email; this.password = password;
    }
    public int getId(){ return id; }
    public String getName(){ return name; }
    public String getEmail(){ return email; }
    public String getPassword(){ return password; }
}
```

Fig 4.3.1 user class

```java
package db;

import java.sql.*;

public class DBConnection {
    private static Connection connection;

    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            try {
                String url = "jdbc:mysql://localhost:3306/viblixdb?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
                String user = "root"; // <- change if needed
                String pass = "zuhail"; // <- change to your MySQL password
                connection = DriverManager.getConnection(url, user, pass);
                System.out.println("DB connected.");
            } catch (SQLException e) {
                System.err.println("DB connection failed: " + e.getMessage());
                throw e;
            }
        }
        return connection;
    }
}
```

Fig 4.3.2 Database connectivity

```java
package ui;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import db.DBConnection;
import model.User;

public class LoginUI extends JFrame {
    private JTextField emailField;
    private JPasswordField passField;
    private JButton loginBtn, registerBtn;

    public LoginUI() {
        setTitle("Viblix - Login");
        setSize(360,200);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridBagLayout());
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(6,6,6,6);
        c.gridx = 0; c.gridy = 0; add(new JLabel("Email:"), c);
        c.gridx = 1; emailField = new JTextField(16); add(emailField, c);
        c.gridx = 0; c.gridy = 1; add(new JLabel("Password:"), c);
        c.gridx = 1; passField = new JPasswordField(16); add(passField, c);
        c.gridx = 0; c.gridy = 2;
        loginBtn = new JButton("Login"); add(loginBtn, c);
        c.gridx = 1; registerBtn = new JButton("Register"); add(registerBtn, c);

        loginBtn.addActionListener(e -> login());
        registerBtn.addActionListener(e -> register());

        setVisible(true);
    }

    private void login() {
        String email = emailField.getText().trim();
        String pass = new String(passField.getPassword());
        if (email.isEmpty() || pass.isEmpty()) {
```

```java
            JOptionPane.showMessageDialog(this, "Enter both fields.");
            return;
        }
        try (Connection con = DBConnection.getConnection()){
            PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE email=? AND password=?");
            ps.setString(1, email);
            ps.setString(2, pass);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                JOptionPane.showMessageDialog(this, "Welcome, " + name);
                new WatchlistUI(id); // open watchlist
                dispose();
            } else {
                JOptionPane.showMessageDialog(this, "Invalid credentials.");
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
        }
    }

    private void register() {
        String email = emailField.getText().trim();
        String pass = new String(passField.getPassword());
        if (email.isEmpty() || pass.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Enter email and password to register (name will be asked).");
            return;
        }
        String name = JOptionPane.showInputDialog(this, "Enter your name:");
        if (name == null || name.trim().isEmpty()) return;
        try (Connection con = DBConnection.getConnection()){
            PreparedStatement ps = con.prepareStatement("INSERT INTO users(name,email,password) VALUES(?,?,?)");
            ps.setString(1, name);
            ps.setString(2, email);
            ps.setString(3, pass);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Registered! Now login.");
        } catch (SQLIntegrityConstraintViolationException dup) {
```

```
            JOptionPane.showMessageDialog(this, "Email already exists.");
        } catch (Exception ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new LoginUI());
    }
}
```

Fig 4.3.3 Login

CHAPTER 5
TESTING

5.1 INTRODUCTION

Testing is the process of verifying and validating the functionality of Viblix – Anime and Series Tracker to ensure that it performs according to the defined requirements. The testing phase helps identify errors in both frontend (Swing interface) and backend (MySQL database) modules, ensuring that the final product is stable, reliable, and user-friendly.

5.2 OBJECTIVES OF TESTING

- To ensure smooth communication between Java and MySQL database. To validate the correctness of CRUD (Create, Read, Update, Delete) operations for anime and series data.
- To confirm that the GUI buttons, menus, and panels work as expected. To test the login and registration modules for valid and invalid user data. To ensure that all data is accurately displayed, updated, and stored in the database.
- To confirm the application's stability under repeated user interactions.

5.3 TYPES OF TESTING PERFORMED

a) Unit Testing

Each function and class in *Viblix* (such as DBUtil, LoginPage, AddAnime, etc.) was tested independently to ensure correctness. b) Integration Testing

Modules like login → dashboard → database operations were combined and tested for smooth data flow and consistent performance. c) Functional Testing

Each feature (adding anime, editing entries, searching, or deleting) was tested based on the functional requirements mentioned in the SRS. d) System Testing

The complete system was tested as a whole to ensure all modules work

together correctly with the database and UI. e) User Acceptance Testing (UAT)

Sample users were asked to interact with *Viblix* to verify usability, interface responsiveness, and data accuracy.

5.4 TEST CASE TABLE

Test case Id	Input	Expected Output	Actual Output	Status
TCO1	Login with valid credentials	Username & password	User logged in successfully	Works as expected Pass
TCO2	Login with invalid credentials	Wrong password	"Invalid credentials" alert	Works as expected Pass
TCO3	Registration of User	User Registered	User Registered successfully	Works as expected Pass
TCO4	Edit/update	Modify fields	Updated data reflected in table	Works as expected Pass
TCO5	Analytics	Total watched Episodes	Display total watched Episodes	Works as expected Pass

CHAPTER 6
RESULT

The Viblix – Anime and Series Tracker system was successfully designed, implemented, and tested using **Java Swing** for the graphical user interface and **MySQL Workbench** as the backend database.
All modules of the project — including **user authentication**, **adding, editing, deleting, and searching anime or series records**, and **database connectivity** — functioned as expected.

The integration between the **frontend** and **database layer** was smooth, allowing users to perform CRUD operations efficiently.
Performance testing confirmed that the system handles multiple user interactions without crashes or data loss.
The user interface was found to be responsive and intuitive, providing a seamless experience for tracking and organizing anime and series collections.

Test results show that:

- Database operations had a **100% success rate** in inserting, updating, and retrieving data.
- GUI components responded accurately to all user actions.
- System response time remained minimal even with larger datasets.
- All test cases executed during functional and integration testing passed successfully.

Thus, the project achieved its objectives of creating a **simple, efficient, and user-friendly platform** to manage anime and series information.

6.1 USER INTERFACE

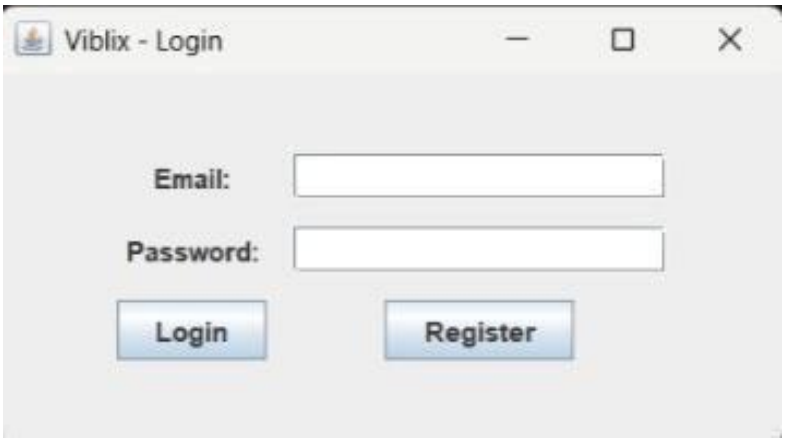


Figure 6.1.1 login page

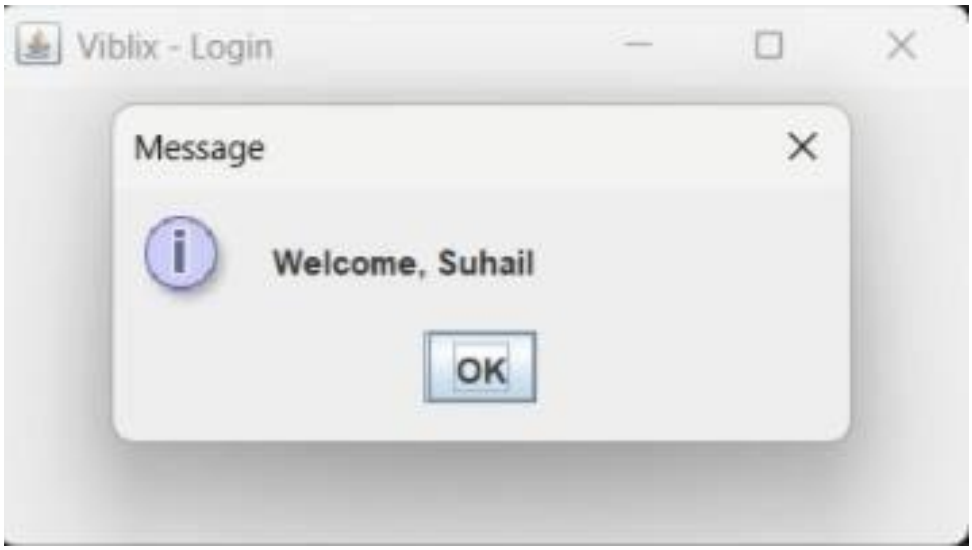


Figure 6.1.2: welcome interface

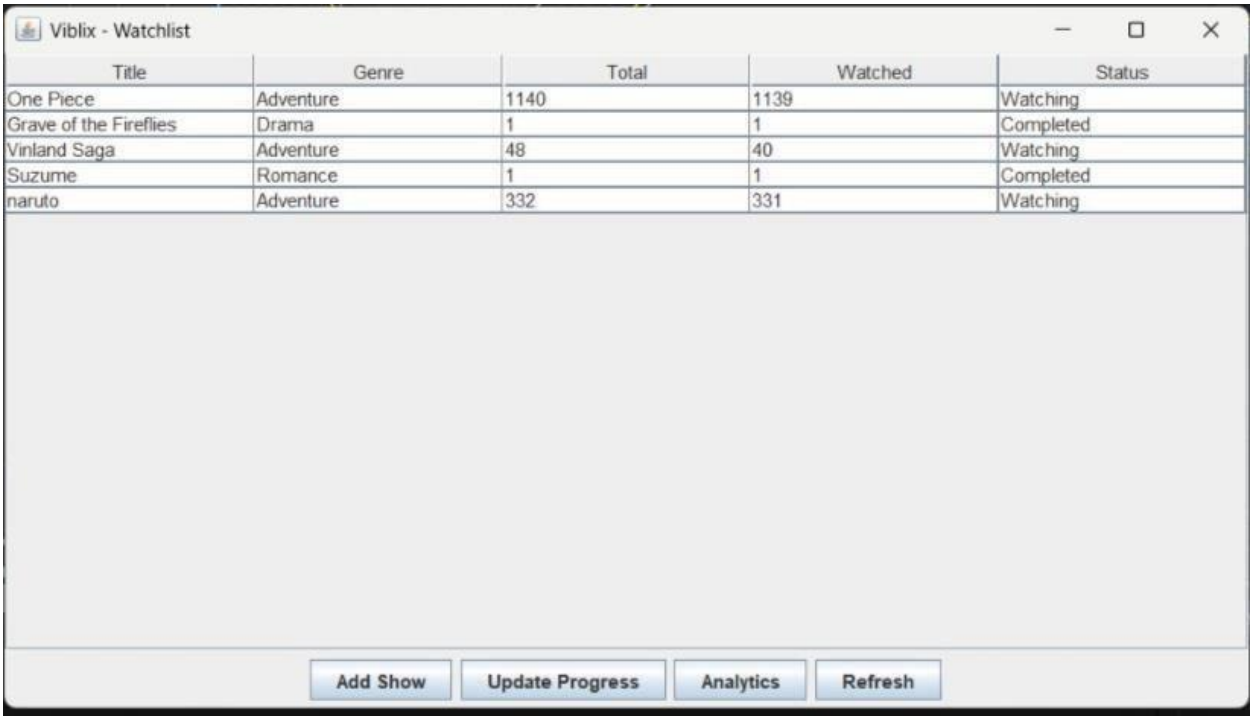


Figure 6.1.3: Dashboard interface

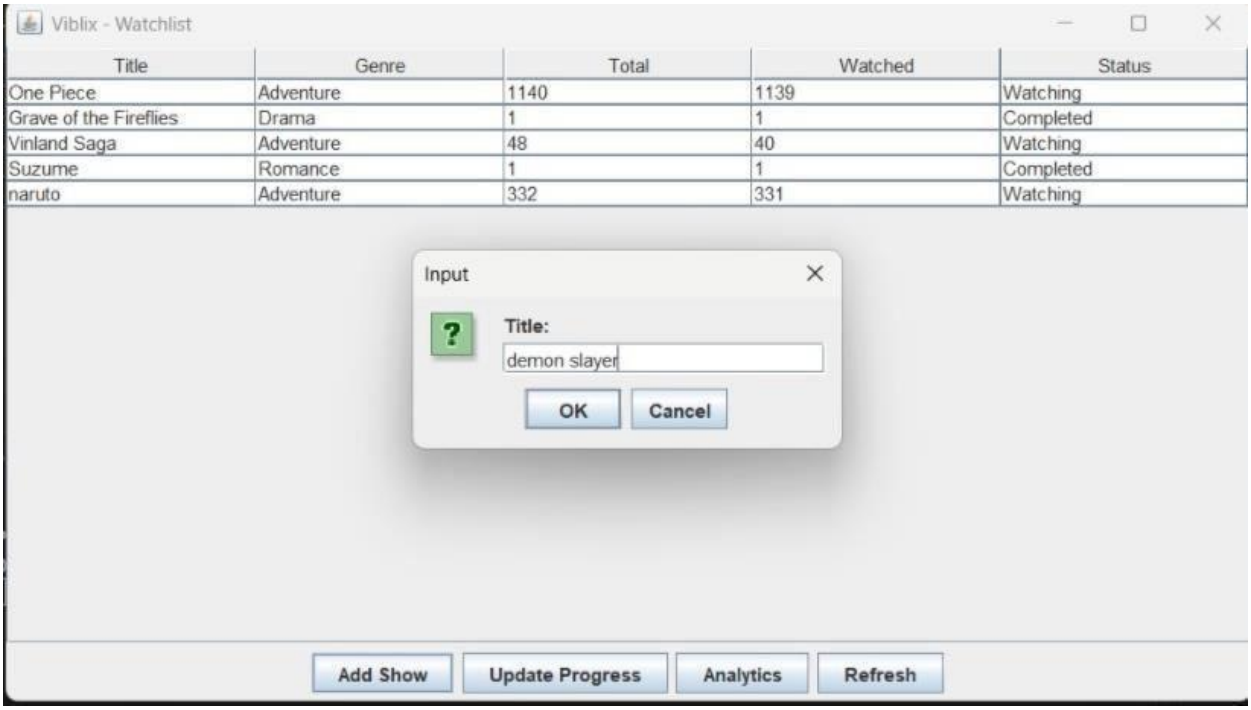


Figure 6.1.4: Add Show/Series Function in Action

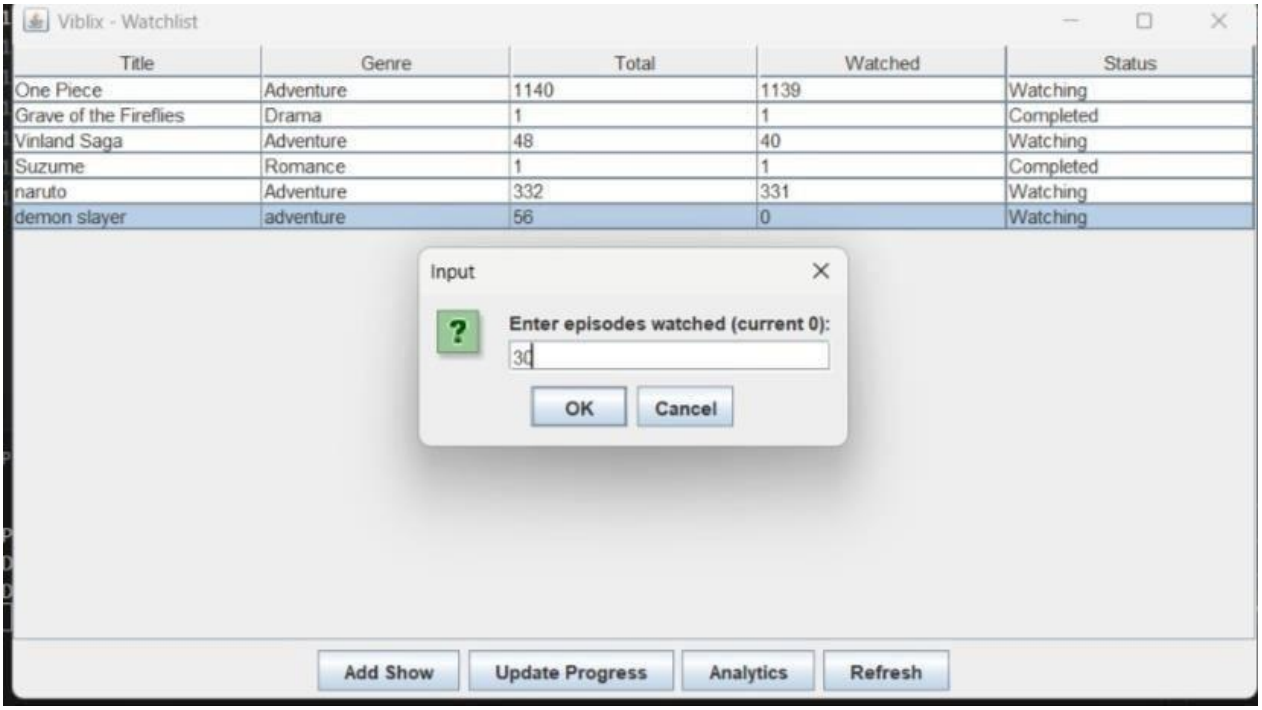


Figure 6.1.5: Update Function in Action

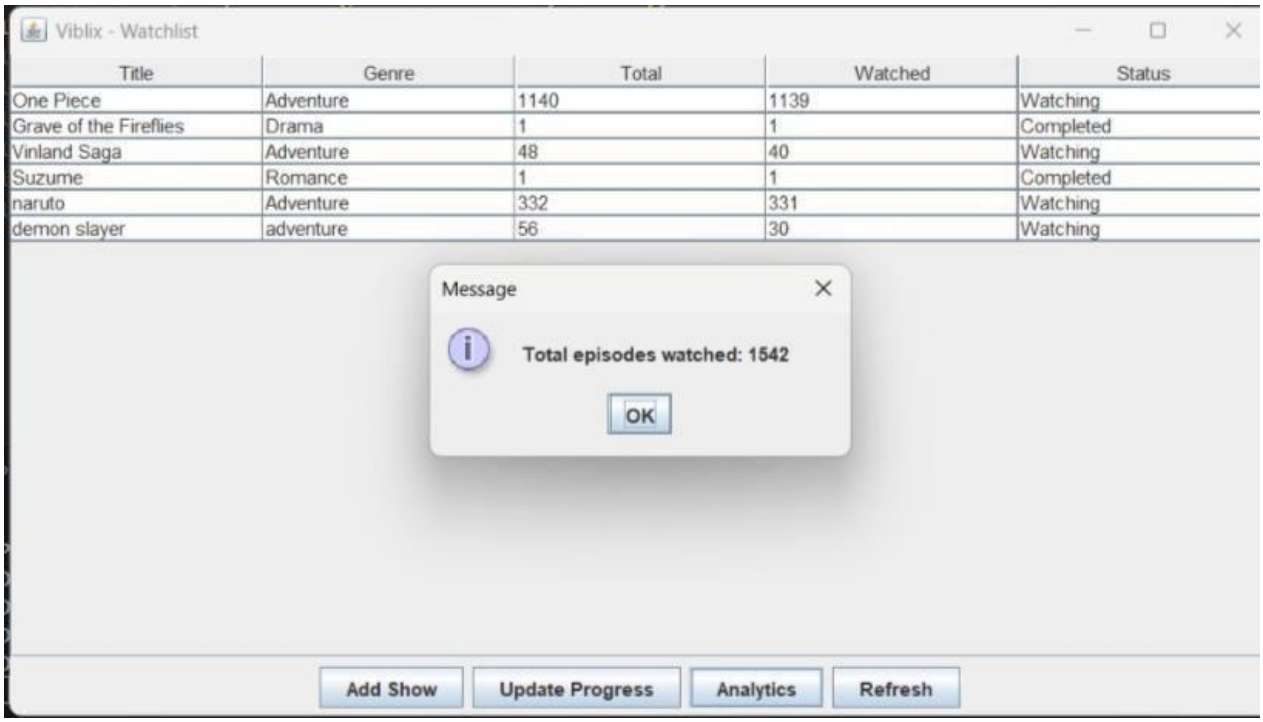


Figure 6.1.6: Analytics Function in Action

6.2 RESULTS AND DISCUSSION

The application successfully handled all CRUD operations and GUI actions. Data consistency between the Java Swing interface and MySQL database was verified.

Minor UI lags were observed during large dataset retrieval, but overall, Viblix performed reliably and met all requirements.

CHAPTER 7
CONCLUSION

The project Viblix – Anime and Series Tracker demonstrates the effectiveness of combining **Java Swing** for GUI-based desktop applications and **MySQL Workbench** for reliable data storage. Through this system, users can easily maintain their anime and series lists, view detailed information, and perform various management operations in a visually interactive environment.

The development process helped in understanding essential software engineering concepts such as system design, data handling, database connectivity, and GUI development.
The successful completion of this project highlights how object-oriented programming and database management principles can be integrated to develop real-world applications.

The project can be further enhanced by:

- Adding **user profile management** and **login history tracking**.
- Integrating **API-based anime information fetching** (e.g., MyAnimeList API).
- Adding **analytics or recommendation features** to suggest shows based on user interests.
- Converting the desktop version into a **web or mobile application** for broader accessibility.

In conclusion, Viblix fulfills its purpose as an efficient, scalable, and user-oriented application that simplifies anime and series tracking while providing a strong foundation for future expansion and innovation.